Timoni + CUE, alongside Flux
Author → render → reconcile

Tool	Responsibility
CUE + Timoni	Define constrained inputs; build and package objects.
Kustomize	Transform YAML using bases, patches and replacements.
Flux	Continuously reconcile desired state in Kubernetes.

Contracts describe allowed inputs before resources are rendered.

Sources: Timoni · Kustomize · Flux

CUE’s creator: Marcel van Lohuizen

- Founding contributor to **Borg** at Google.
- Created **BCL**, the Borg Configuration Language, later generalized into GCL.
- Created **CUE**, drawing on experience with large configurations.

Borg / BCL → GCL → CUE

The design idea: combine **data and constraints** in one language.

Sources: Marcel’s biography · CUE history

Types and values share one model

A constraint describes a **set of allowed values**.

```
replicas: *2 | (int & ⩾1 & ⩽10)
```

- `int & ⩾1 & ⩽10`: integers in the intersection of these rules.
- `|` offers alternatives; `*2` marks the preferred default.

No input → **2**	Input `3` → **3**	Input `0` → **error**

Sources: Constraints · Defaults

Composition means satisfying both

Reusable definition + concrete input

```
#Config: {
  replicas: int & ⩾1 & ⩽10
}

a: #Config & {replicas: 3}
b: {replicas: 3} & #Config
```

Unification

Both `a` and `b` resolve to:

```
replicas: 3
```

`A & B = B & A`

Conflicting information produces an error.

`#Config` names reusable structure and constraints.

Sources: Unification · Definitions · Composition properties

Transform YAML or constrain inputs

Kustomize · kustomization.yaml excerpt

```yaml
resources:
- ../base
replicas:
- name: demo
  count: 3
```

CUE · definition + input

```
#Config: {
  replicas: *2 | (int & ⩾1 & ⩽10)
}
values: #Config & {replicas: 3}
```

Input	Kustomize result	CUE module result
`3`	Render `3`	Accept `3`
`0`	Render `0`	Reject: outside `1–10`

Kustomize can be paired with separate schema and policy checks.

Sources: Kustomize operations · Offline comparison

Timoni packages CUE modules

- **Interface:** values, defaults and constraints in a module.
- **Schemas:** vendor Kubernetes API and CRD definitions for validation.
- **Distribution:** versioned CUE source modules in OCI registries.

Build and validate the local WebApp module

```
timoni build rgd-demo ./example-rgd -n demo
timoni mod vet ./example-rgd --name rgd-demo -n demo
```

Build produces YAML. Timoni also offers CLI deployment and inventory.

Sources: Timoni concepts · Modules · CRD schemas

Use shared CUE structure for a kro API

CUE · schema + shared conventions

```
#RGD:
  rgdv1.#ResourceGraphDefinition & {
    spec: schema: {
      group: "platform.example"
      scope: "Namespaced"
    }
  }
#WebAppRGD: #RGD & {
  metadata: name: "web-app"
  // API + resource templates
}
```

Rendered RGD · schema/template excerpt

```
schema:
  kind: WebApp
  spec:
    replicas: >-
      integer | default=2
      minimum=1 maximum=10
resources:
- id: webDeployment
  template:
    kind: Deployment
    spec:
      replicas: >-
        ${schema.spec.replicas}
```

Timoni sets the API default **2**; kro uses each instance's value.

Sources: kro v0.9.4 · Complete module, including vendored schema · Flux values handoff

External values and Flux substitution

Flux delivery pattern

Bicep / CI → ConfigMaps / Secrets → Flux → Kubernetes

Flux Kustomization.spec · excerpt

```yaml
postBuild:
  substituteFrom:
  - kind: ConfigMap
    name: infra-values
  - kind: Secret
    name: infra-secrets
```

Consumer validation

- Flux substitutes values after the Kustomize build.
- CUE checks concrete input values against a contract.
- Runtime can read cluster values into a Timoni Bundle.

Validate producer outputs or resolve inputs before checking the consumer contract.

Sources: Bicep outputs · Flux substitution · Values handoff

Runtime selects resources and fields

Runtime queries · excerpt from runtime.cue

```
runtime: values: [{
  query: "k8s:v1:ConfigMap:demo:infra-values"
  for: "DEMO_REPLICAS": "obj.data.replicas"
}, {
  query: "k8s:v1:Secret:demo:infra-secrets"
  for: "DEMO_TOKEN": "obj.data.token"
}]
```

- `query`: API version, kind, namespace and object name.
- `for`: named input → expression selecting a field.
- Bundle attributes choose how each input is parsed.

Object field → named Runtime input → typed Bundle value

Sources: Runtime queries and mappings · Complete runtime.cue

Bundle inputs: bind, then constrain

bundle.cue · contract, bindings and instance values (metadata omitted)

```
#InfraValues: {
  replicas: int & ⩾1 & ⩽10
  token:    string & ≠""
}
bundle: {
  _replicas: number @timoni(runtime:number:DEMO_REPLICAS)
  _token:    string @timoni(runtime:string:DEMO_TOKEN)
  instances: web: values: #InfraValues & {
    replicas: _replicas
    token:    _token
  }
}
```

Parse Runtime inputs → unify with the contract → supply module values

Sources: Bundles · Runtime bindings · Hidden fields · Complete bundle.cue

Check the contract offline

From example-runtime/ · synthetic inputs

```
DEMO_REPLICAS=3 DEMO_TOKEN=demo-only \
  timoni bundle vet -f bundle.cue -r runtime.cue \
  --offline --kube-context=minikube
```

Input	Contract result
Replicas `"3"`, nonempty token	Accept integer `3` and token
Replicas `0`, `11`, `3.5` or `three`	Reject range, type or parsing
Empty or missing token	Reject required string

Runtime reads on invocation. Input changes need a new run; Flux reconciles separately.

Sources: Offline Runtime inputs · Timoni and Flux · Reproduce the checks

Capabilities and operating requirements

Capabilities

- Reusable CUE contracts and composition.
- Module builds and vendored schema checks.
- OCI modules and Runtime value binding.

Operating requirements

- CUE, schema and toolchain maintenance.
- Concrete inputs at the validation boundary.
- A trigger for each Runtime execution.

Timoni builds and applies when invoked; Flux continuously reconciles.

Examples: Deployment · Kustomize comparison · WebApp RGD · Runtime inputs